

Course Outline: API Documentation

Title: API Documentation **Learnpack:** + Documentación de APIs **Prerequisite:** Building a Python API to serve the frontend (same day, Skill 19) **Target Audience:** Beginner developers who have just built their first CRUD API with FastAPI and need to learn how to document it for humans and AI agents **Total Lessons:** 11
Estimated Total Length: ~5,500 words **Format:** Conceptual (0% hands-on)

Course Description

You just built an API — congratulations. But if no one knows how to use it, it might as well not exist. API documentation is the bridge between what your code does and what other developers (or AI agents) can actually do with it.

In this course, you will learn what makes API documentation useful, how to write it for both human developers and machine consumers, and how FastAPI can generate most of it for you automatically — if you set things up correctly. By the end, you will understand that documentation is not an afterthought; it is part of the product.

This course connects directly to what comes next: in the following lesson, you will build AI agents that use tools — and those tools are described through documentation. If the docs are bad, the agent fails.

Course Philosophy

- This course emphasizes:
- **Documentation as product:** Docs are not a chore after coding — they are part of delivering a working API.
 - **Two audiences, one source of truth:** Every API serves humans and increasingly machines. Good documentation serves both without duplication.
 - **Path of least resistance:** FastAPI already generates docs for you. Learn to enrich what is generated rather than writing everything from scratch.
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - What Is API Documentation?	2	~1,000
02 - Writing Documentation for Humans and Agents	3	~1,600
03 - Auto-Generated Documentation in FastAPI	2	~1,100
04 - Assessment	1	~700
05 - Wrapping Up	1	~450

Section	Lessons	Estimated Words
Total	11	~5,500

Section 00: Welcome

00.0 Why Document Your API? 📖 (~450 words)

Learning Objectives:

- Recognize that an undocumented API is effectively unusable for other developers and AI agents
- Identify the two primary audiences of API documentation: humans and machines

Content Outline:

- The scenario: you built an API, someone else needs to use it — what do they need to know?
- What happens when documentation is missing or wrong (wasted time, wrong integrations, broken agents)
- Brief preview: documentation for humans vs documentation for AI agents
- Why this matters now: the next skill involves building AI agents that rely on well-documented tools

Transitions: This lesson sets the stage. Next, we break down what API documentation actually contains and what separates good docs from bad ones.

Key Principles:

- An API without documentation is an API nobody can use
- Documentation is not optional — it is part of delivering a working product

Examples:

- Trying to use a public API (like a weather API) with no docs vs one with clear examples — the difference in time to first successful request
- An AI agent that fails to call a tool because the description was vague or the parameters were not explained

Section 01: What Is API Documentation?

01.0 The Anatomy of Good API Documentation 📖 (~500 words)

Learning Objectives:

- List the essential components of API documentation (endpoint URL, method, parameters, request/response examples, status codes, authentication)
- Distinguish between reference documentation and guides/tutorials

Content Outline:

- The minimum pieces every documented endpoint needs: URL, HTTP method, description of what it does, parameters (path, query, body), example request, example response, possible error codes

- Reference docs vs guides: reference tells you "what exists," guides tell you "how to accomplish X"
- Where documentation typically lives (dedicated docs pages, README files, auto-generated tools)

Transitions: Now that you know what the pieces are, the next lesson focuses on how to tell the difference between docs that actually help and docs that waste your time.

Key Principles:

- Good documentation answers: "What does this endpoint do? What do I send? What do I get back? What can go wrong?"
- Reference and guides serve different purposes — most APIs need at least the reference part

Examples:

- A well-documented POST /users endpoint showing: URL, method, body schema, 201 response, 400 and 409 error responses
 - A poorly documented endpoint that only says "Creates a user" with no parameter info
-

01.1 Reading and Navigating Real API Docs 📖 (~500 words)

Learning Objectives:

- Navigate a real API documentation page and locate the information needed to make a request
- Identify what is missing when documentation is incomplete

Content Outline:

- Walking through a real-world API documentation page (e.g., Stripe, GitHub, or a weather API)
- How to find: base URL, authentication method, specific endpoints, request/response formats
- Spotting gaps: what happens when docs don't mention error codes, rate limits, or required headers
- The habit of reading docs before writing code — not after something breaks

Transitions: You can now read and evaluate documentation. In the next section, you will learn how to write it — first for human developers, then for AI agents.

Key Principles:

- Always read the documentation before making your first request
- If the docs are confusing, the problem is the docs — not you

Examples:

- Finding the authentication section in GitHub's API docs and identifying that a Bearer token is needed
 - An API that documents GET /items but does not mention that it requires a query parameter — the developer discovers this only after getting a 400 error
-

Section 02: Writing Documentation for Humans and Agents

02.0 Describing Endpoints Clearly for Humans 📖 (~550 words)

Learning Objectives:

- Write a clear, complete description for an API endpoint that another developer can use without asking questions
- Apply best practices for documenting parameters, request bodies, and responses

Content Outline:

- Writing endpoint descriptions that answer "what" and "why," not just the HTTP method
- Documenting parameters: name, type, required vs optional, default values, constraints
- Request body documentation: schema, field descriptions, example payloads
- Response documentation: success responses with example data, error responses with status codes and messages
- The importance of showing concrete examples — developers copy examples more than they read descriptions

Transitions: You now know how to write docs that humans can use. But there is a second audience that is becoming just as important: AI agents and LLMs. The next lesson covers what they need.

Key Principles:

- Show, don't just tell — example requests and responses are more useful than paragraphs of explanation
- Document the errors, not just the happy path

Examples:

- A well-documented PATCH /contacts/{id} endpoint: description, path param (id: integer, required), body schema (name: string optional, email: string optional), 200 response with updated object, 404 response when contact not found
 - A bad example: "Updates a contact" with no parameter info and no response examples
-

02.1 Structuring Docs for AI Agents and LLMs 📖 (~550 words)

Learning Objectives:

- Explain why AI agents need well-structured API documentation to function correctly
- Identify the key differences between documentation optimized for humans vs for machines

Content Outline:

- Why this matters now: AI agents use tool descriptions to decide which endpoint to call and how — vague docs mean wrong decisions
- What agents need: precise parameter names and types, explicit required/optional markers, unambiguous descriptions, structured schemas
- Human docs can rely on context and inference; agent docs cannot — machines take descriptions literally
- The connection to the next skill (Skill 20): students will build agents that select and call tools based on descriptions — those descriptions are documentation
- Practical differences: a human reads "pass the user ID" and knows where to put it; an agent needs to know it is a path parameter of type integer named "user_id"

Transitions: You now understand both audiences. Before we look at how FastAPI handles this automatically, let's cover the mistakes that make documentation fail for both humans and agents.

Key Principles:

- If an AI agent cannot understand your docs, your docs are not precise enough
- Structured, machine-readable documentation (like OpenAPI schemas) serves both humans and agents

Examples:

- A tool description for an agent: "Search contacts by name. Parameters: query (string, required) — the search term. Returns: array of contact objects with id, name, email." — the agent can use this
 - A vague tool description: "Gets contacts" — the agent does not know what parameters to pass or what to expect back
-

02.2 Common Documentation Anti-Patterns 📖 (~500 words)

Learning Objectives:

- Recognize the most common documentation mistakes that make APIs hard to use
- Explain why each anti-pattern is harmful and what to do instead

Content Outline:

- **"Self-documenting code" excuse:** Believing that clean code eliminates the need for docs — it does not; external consumers cannot read your source code
- **Stale documentation:** Docs that described the API six months ago but have not been updated since — worse than no docs because they actively mislead
- **Missing error documentation:** Only documenting the happy path (200 OK) and leaving developers to guess what 400, 404, or 500 responses look like
- **Copy-paste descriptions:** Every endpoint says "Returns data" or "Processes request" — technically true, completely useless
- **Overly verbose docs:** Three paragraphs explaining what a GET endpoint does when a one-line description and an example would suffice
- The correct approach for each: keep docs close to code (so they stay in sync), document errors alongside successes, write specific descriptions, prefer examples over prose

Transitions: Now that you know what to do and what to avoid, let's look at how FastAPI takes care of a large part of this work automatically — and how to make the most of it.

Key Principles:

- Stale documentation is more dangerous than missing documentation — it creates false confidence
- The best docs stay in sync with the code because they are generated from the code

Examples:

- An endpoint documented as "Handles user data" — a developer still has no idea if it creates, reads, updates, or deletes users

- A POST endpoint that documents the 201 response but not the 422 validation error — the developer sends malformed data and gets an unexpected error with no guidance
-

Section 03: Auto-Generated Documentation in FastAPI

03.0 How FastAPI Generates Docs Automatically 📖 (~550 words)

Learning Objectives:

- Explain how FastAPI uses Python type hints and Pydantic models to generate OpenAPI documentation
- Identify where to access the auto-generated docs (Swagger UI at /docs, ReDoc at /redoc)

Content Outline:

- FastAPI's built-in documentation: Swagger UI (/docs) and ReDoc (/redoc) — available out of the box with zero configuration
- How it works: FastAPI reads your route decorators, function signatures, type hints, and Pydantic models to build an OpenAPI specification (a machine-readable JSON schema)
- What gets generated automatically: endpoint URLs, HTTP methods, parameter names and types, request body schemas from Pydantic models, response status codes
- The OpenAPI spec as the single source of truth: both the Swagger UI and any AI agent can read this same spec
- Connection to the previous lesson: this is how documentation stays in sync with code — because it IS the code

Transitions: FastAPI generates a solid foundation, but the default output is often too minimal. The next lesson shows how to enrich it with descriptions, examples, and metadata that make your docs truly useful.

Key Principles:

- FastAPI generates documentation from your code — if your types are correct, your docs are already halfway done
- The OpenAPI specification serves both interactive docs for humans and structured schemas for AI agents

Examples:

- A FastAPI endpoint with typed parameters and a Pydantic model: the auto-generated Swagger UI shows the schema, parameters, and a "Try it out" button — all without writing a single line of documentation
 - The same endpoint without type hints: Swagger shows the route exists but cannot display parameter types or body schema — the docs are nearly useless
-

03.1 Enriching FastAPI's Auto-Generated Docs 📖 (~550 words)

Learning Objectives:

- Add descriptions, examples, and response documentation to FastAPI endpoints to improve the auto-generated docs

- Use Pydantic model configuration (Field descriptions, examples) to enrich the OpenAPI schema

Content Outline:

- Adding descriptions to routes: the `summary` and `description` parameters in route decorators
- Documenting response models: using `response_model` to define what the endpoint returns, and `responses` parameter for error cases (404, 422)
- Enriching Pydantic models: using `Field(description=..., example=...)` so that the generated schema includes helpful metadata
- Tags for grouping endpoints: organizing the Swagger UI into logical sections (e.g., "Contacts", "Authentication")
- The result: documentation that is generated from code, stays in sync automatically, and is rich enough for both human developers and AI agents to use effectively

Transitions: You now know how to write documentation, avoid common mistakes, and leverage FastAPI's auto-generation. Time to test your understanding.

Key Principles:

- The best documentation strategy is enriching auto-generated docs, not replacing them with manual writing
- A few minutes adding descriptions and examples to your Pydantic models saves hours of confusion for every consumer of your API

Examples:

- A Pydantic model with bare fields (`name: str, email: str`) vs one with enriched fields (`name: str = Field(description="Full name of the contact", example="Jane Doe")`) — the Swagger UI output is dramatically different
- An endpoint using `responses={404: {"description": "Contact not found"}}` — the docs now show what happens on errors, not just success

Section 04: Assessment

04.0 API Documentation Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of API documentation best practices for humans and AI agents
- Identify correct and incorrect documentation approaches in realistic scenarios

Question Format: Multiple choice

Questions:

Q1. A developer documents a POST /users endpoint with only the description: "Creates a user." A teammate tries to use the endpoint but does not know what fields to include in the request body. What is the most important missing piece of documentation?

- A) The HTTP method

- B) The request body schema with field names, types, and required/optional markers
- C) The server URL
- D) The programming language used to build the API

Correct: B — Without the body schema, the consumer cannot construct a valid request.

Q2. Why is stale (outdated) documentation considered more dangerous than missing documentation?

- A) Because it takes up server storage space
- B) Because it actively misleads developers into making incorrect requests
- C) Because search engines penalize outdated content
- D) Because it slows down the API response time

Correct: B — Stale docs create false confidence; developers trust them and waste time debugging requests that follow outdated specs.

Q3. An AI agent receives this tool description: "Gets contacts." The agent needs to search contacts by name. What is the most likely outcome?

- A) The agent will successfully search contacts by name
- B) The agent will not know what parameters to pass and will likely call the endpoint incorrectly or not at all
- C) The agent will automatically read the source code to find the parameters
- D) The agent will ask the user for clarification

Correct: B — Agents take descriptions literally and cannot infer undocumented parameters.

Q4. How does FastAPI generate its interactive documentation at /docs?

- A) Developers must write a separate YAML file manually
- B) FastAPI reads route decorators, type hints, and Pydantic models to build an OpenAPI specification automatically
- C) FastAPI copies documentation from a README file in the project
- D) The documentation is hosted by a third-party service and must be configured externally

Correct: B — FastAPI uses the code itself (types, models, decorators) to generate the OpenAPI spec that powers Swagger UI.

Q5. A Pydantic model defines `name: str` and `email: str` with no additional metadata. What will be missing from the auto-generated Swagger documentation?

- A) The field names
- B) The field types
- C) Descriptions, examples, and validation constraints for each field
- D) The endpoint URL

Correct: C — Bare type hints give you names and types, but no descriptions or examples. Adding `Field(description=..., example=...)` fixes this.

Q6. Which of the following is the best strategy for keeping API documentation accurate over time?

- A) Assign one person to manually update a wiki page after each deployment
- B) Enrich auto-generated documentation (like FastAPI's) so docs stay in sync with the code
- C) Write all documentation at the end of the project, once the API is stable
- D) Only document endpoints that external clients use, skip internal ones

Correct: B — Auto-generated docs stay in sync because they come from the code itself. Enriching them with descriptions and examples is the path of least resistance.

Evaluation Criteria:

- 6/6: Excellent understanding of API documentation principles
 - 4-5/6: Solid understanding with minor gaps
 - Below 4: Review the course material, focusing on the sections where questions were missed
-

Section 05: Wrapping Up

05.0 Your APIs Now Speak for Themselves (~450 words)

Content Outline:

- Course recap: you learned what API documentation is, why it matters for both humans and AI agents, how to write it effectively, what mistakes to avoid, and how FastAPI generates most of it from your code
- Connection to the bigger picture: in the very next skill (Skill 20), you will build AI agents that use tools — and those tools are defined by documentation. Everything you learned here directly enables what comes next
- Next steps: as you build more endpoints, make it a habit to add descriptions and examples to your Pydantic models and route decorators — the few extra seconds pay off every time someone (or some agent) uses your API
- Resources: FastAPI official documentation on metadata and docs customization, the OpenAPI specification reference, Swagger UI documentation
- Encouragement: documentation might not feel as exciting as writing code, but it is what turns your code from "something that works on my machine" into "something the world can use." Your APIs now speak for themselves — and that includes speaking to AI.

Note: This is conclusion only — no theory, exercises, or assessment.
